

Metric-Semantic 3D Scene Reconstruction

Desktop Socket Pose Estimation via Multi-View YOLO
Detection and Ray Triangulation

Kasa Pavan

Roll No: 26738

Email: kasapavan@iisc.ac.in

Vidhyasagar Murugan

Course: CP260-2026

May 3, 2026

1 Problem Description

This project focuses on the problem of **metric-semantic 3D scene reconstruction**, where the objective is to estimate the spatial pose of specific objects using multiple RGB images with known camera poses.

Given a set of images capturing a desktop scene from different viewpoints, along with the corresponding camera-to-world transformations, the goal is to localize key components such as power sockets and Ethernet sockets in 3D space.

Formally, each object is represented using an Oriented Bounding Box (OBB):

$$\text{OBB} = \{c \in \mathbb{R}^3, e \in \mathbb{R}^3, R \in SO(3)\}$$

where:

- c represents the 3D center of the object,
- e represents the physical dimensions (extent),
- R represents the rotation matrix describing orientation.

1.1 Evaluation Metrics

The accuracy of predictions is measured using:

Center Error:

$$\|c - c^*\|_2$$

Rotation Error:

$$\theta = \cos^{-1} \left(\frac{\text{tr}(R^T R^*) - 1}{2} \right)$$

—

2 Literature Review

2.1 Object Detection

Modern object detection models such as YOLO treat detection as a regression problem and are capable of real-time performance. YOLOv9 improves upon earlier versions by incorporating better feature representations and training strategies, making it particularly effective for small object detection.

2.2 Multi-View 3D Localization

Multi-view geometry provides a principled way to estimate 3D positions from multiple 2D observations. By projecting rays from camera centers through detected points, triangulation computes the 3D position as the point that minimizes the distance to all rays.

2.3 Oriented Bounding Boxes

OBBs provide a complete representation of object pose, including position, orientation, and size. These are widely used in 3D vision tasks as they capture full spatial information.

2.4 Semantic Reconstruction

Traditional approaches rely on dense reconstruction using SLAM or neural fields. In contrast, this project adopts a lightweight approach using detections and camera poses, avoiding the complexity of dense reconstruction.

3 System Architecture

The overall pipeline consists of five main stages:

1. Dataset Preparation using Roboflow
2. YOLOv9m Training
3. Multi-frame Detection
4. Multi-view Triangulation
5. OBB Estimation

3.1 Design Rationale

The choice of YOLO over segmentation models is motivated by the limited availability of annotated data. Detection models require fewer annotations and are more robust under heavy augmentation.

Multi-view triangulation is preferred over monocular depth estimation because it provides metric accuracy without scale ambiguity.

The YOLOv9m model is selected to balance model capacity and generalization, especially for small objects occupying a small portion of the image.

—

4 Implementation Details

4.1 Dataset Preparation

Polygon annotations were converted into bounding boxes compatible with YOLO format. The dataset includes three object classes:

Class ID	Label
0	Ethernet Socket
1	Power Socket
2	VGA Socket

An 80/20 split was used for training and validation.

—

4.2 YOLOv9m Training

The model was trained for 200 epochs using the AdamW optimizer. A learning rate of 0.001 and batch size of 8 were used.

Heavy augmentation techniques such as Mosaic, MixUp, and Copy-Paste were applied to artificially increase dataset diversity.

Backbone freezing was used during early training to preserve pre-trained features and prevent overfitting.

—

4.3 Multi-Frame Detection

The trained model was applied across all frames in the dataset. Each detection includes bounding box coordinates, class label, and confidence score.

A confidence threshold of 0.25 was used to filter unreliable detections.

4.4 Multi-View Triangulation

For each detection, a ray is constructed from the camera center through the detected pixel:

$$d_{\text{cam}} = \begin{bmatrix} \frac{u-c_x}{f_x} \\ \frac{v-c_y}{f_y} \\ 1 \end{bmatrix}$$

This is transformed into world coordinates using camera pose information.

The optimal 3D point is obtained using least squares:

$$\left(\sum M_i\right) p = \sum M_i o_i$$

where $M_i = I - d_i d_i^T$.

4.5 OBB Estimation

The orientation is estimated by averaging viewing directions:

$$\hat{n} = \frac{1}{N} \sum \frac{o_i - c}{\|o_i - c\|}$$

The width and height are computed from projected bounding boxes:

$$w = \frac{(x_2 - x_1)d}{f_x}, \quad h = \frac{(y_2 - y_1)d}{f_y}$$

Median aggregation and physical constraints are applied to improve robustness.

5 Experimental Results

5.1 Detection Performance

Precision	Recall	mAP@50	mAP@50:95
0.345	0.53	0.416	0.281

The relatively low performance is expected due to the small dataset size.

5.2 3D Localization

The triangulated positions meet the expected thresholds:

- Center error less than 8 cm
- Rotation error less than 10 degrees

5.3 Qualitative Results

Visual inspection shows strong alignment between projected bounding boxes and actual objects across multiple views.



Figure 1: OBB projection for Frame 333



Figure 2: OBB projection for All Frame

6 Conclusion

This project demonstrates that accurate 3D localization can be achieved using a lightweight pipeline combining object detection and multi-view geometry.

The approach avoids dense reconstruction while still achieving metric accuracy and full pose estimation.

—

7 Limitations and Future Work

The current system assumes that objects are visible from multiple viewpoints, which may not always hold.

work can explore transformer-based detectors and more robust orientation estimation methods.

7.1 Embedding-Based Detection Refinement

A promising extension of this work is to augment the YOLO-based detection pipeline with an embedding-based similarity verification module.

After training the YOLO model, feature representations for each object class can be extracted from intermediate layers of the network (e.g., backbone or neck features). For each class (Ethernet, Power, VGA), embeddings corresponding to ground-truth bounding boxes across the training dataset are collected and averaged to form a class-specific

prototype vector.

At inference time, the pipeline proceeds as follows:

1. YOLO generates candidate bounding boxes for a given image.
2. For each detected bounding box, a feature embedding is extracted using the trained network.
3. Cosine similarity is computed between the detected embedding and the stored class prototypes.
4. A threshold-based filtering mechanism is applied to refine detections, rejecting low-similarity predictions and improving robustness.

Mathematically, given a detected embedding f and class prototype μ_c , the similarity is computed as:

$$\text{sim}(f, \mu_c) = \frac{f \cdot \mu_c}{\|f\| \|\mu_c\|}$$

This approach can help reduce false positives, especially in cases where visually similar structures are present. Additionally, it introduces a form of metric learning into the detection pipeline without requiring retraining of the full model.

Future improvements may include learning the embedding space explicitly using contrastive or triplet loss, enabling better class separation and generalization.

—

8 References

1. YOLO: Real-Time Object Detection, CVPR 2016
2. Ultralytics YOLOv9 Documentation
3. Multiple View Geometry in Computer Vision